Estudo Dirigido - TAD Pilha

PUC Minas Contagem - ICEI | Algoritmos e Estrutura de Dados | 2026/2 | Prof. Fábio Leandro Rodrigues Cordeiro

Integrantes (nome completo e matrícula):
1. João Pedro Vital Fortes Matrícula: 1582228
2. João Vitor Matias Gonçalves Matrícula: 1577257
3. Phillipi Garcia da Silveira Matrícula: 1540434

Parte 2 - Rastreio no papel

Sequência executada a partir de `new Pilha(4)` (capacidade 4).

Operação	Retorno	Pilha (base -> topo)	topo
new Pilha(4)	-	[ ] (vazia)	0
empilha(10)	-	[10]	1
empilha(20)	-	[10, 20]	2
desempilha()	20	[10]	1
empilha(30)	-	[10, 30]	2
empilha(40)	-	[10, 30, 40]	3
desempilha()	40	[10, 30]	2
desempilha()	30	[10]	1
empilha(50)	-	[10, 50]	2

Pare e pense. Ao final, o vetor interno está como `item = [10, 50, 40, ?]` e topo = 2. Em `item[2]` está guardado o valor **40**, mas ele **não faz mais parte da pilha**: só os índices de 0 até topo-1 (aqui, 0 e 1) pertencem a ela. O `desempilha` não precisa apagar nada porque quem define o que é válido é o campo `topo`: basta decrementá-lo e a posição passa a ser considerada livre. O valor antigo fica como lixo e será sobrescrito no próximo `empilha`. Isso também mantém as operações em O(1), sem mover elementos.

Parte 4.1 - Rastreio do Bubble Sort com pilhas

Vetor {5, 2, 9, 1, 7}. A começa com 5 na base e 7 no topo. Estado ao final de cada passada (B antes da troca de papéis; depois da troca, B vira a entrada A da passada seguinte).

Passada	carry final	B (base -> topo)	R (base -> topo)
1	9	[1, 7, 2, 5]	[9]
2	7	[2, 5, 1]	[9, 7]
3	5	[1, 2]	[9, 7, 5]
4	2	[1]	[9, 7, 5, 2]
5	1	[ ] (vazia)	[9, 7, 5, 2, 1]

Detalhe da passada 1 (carry começa em 7, que é o topo de A): x=1 -> 7>1, B recebe 1; x=9 -> 7>9 é falso, B recebe 7 e carry=9; x=2 -> 9>2, B recebe 2; x=5 -> 9>5, B recebe 5. R recebe 9. Ao desempilhar R, os valores saem em ordem crescente: **1, 2, 5, 7, 9**.

Parte 4.2 - Resultados dos testes de `bubblePilhas`

Vetor de entrada	Vetor devolvido	comparacoes
{5, 2, 9, 1, 7}	{1, 2, 5, 7, 9}	10

Vetor de entrada	Vetor devolvido	comparacoes
{1, 2, 3, 4, 5} (já ordenado)	{1, 2, 3, 4, 5}	10

Parte 4.3 - Questões

(a) Para n = 5 foram feitas **10 comparações** (4 + 3 + 2 + 1 + 0, uma por passada a menos que a anterior). A expressão geral é (n-1) + (n-2) + ... + 1 = **n(n-1)/2**, que dá 5·4/2 = 10. O Bubble Sort clássico sem otimização com vetor faz exatamente o mesmo número de comparações, n(n-1)/2, e também não depende da ordem inicial dos dados (o vetor já ordenado fez as mesmas 10). Logo o custo em comparações é $O(n^2)$ nos dois casos.
(b) O Bubble Sort com vetor ordena no próprio vetor, com memória extra O(1) (uma variável temporária para a troca). Com pilhas, são três pilhas de capacidade n (A, B e R), ou seja, 3n posições a mais, além da variável carry, e ainda o vetor de saída. O custo extra de memória é **O(n)**.
(c) Deixamos de ter o **acesso direto por índice** e, com ele, a troca de posições (`v[j] <-> v[j+1]`): na pilha só se enxerga o topo. O código compensa mantendo a bolha em uma variável (`carry`) e despejando os demais elementos em outra pilha, comparando carry com o elemento que acabou de sair do topo. Ao esvaziar A para B, a ordem relativa é invertida, e por isso A e B trocam de papel a cada passada. O maior valor de cada passada fica em carry e é empilhado em R.
(d) Na prática, **não vale a pena** quando existe vetor disponível: o algoritmo continua $O(n^2)$ e ainda gasta O(n) de memória extra, sem ganho. O valor está no aprendizado, porque mostra como resolver um problema usando apenas as operações restritas do TAD. A ideia serve quando só se pode acessar os dados pelo topo (uma estrutura LIFO sem acesso por índice) e é preciso ordenar ou reorganizar os elementos, como no desafio de ordenar uma pilha usando uma única pilha auxiliar.

Anexo - Teste de mesa das Partes 1 e 3

Parte 1 - teste do roteiro com `new Pilha(5)`:

Operação	Saída	Pilha (base -> topo)	topo
empilha(1), (3), (5), (7)	-	[1, 3, 5, 7]	4
mostra()	[1, 3, 5, 7]	[1, 3, 5, 7]	4
desempilha()	7	[1, 3, 5]	3
consultaTopo()	5	[1, 3, 5]	3
empilha(8), (9)	-	[1, 3, 5, 8, 9]	5
mostra()	[1, 3, 5, 8, 9]	[1, 3, 5, 8, 9]	5
cheia()	True (topo = 5 = capacidade)	[1, 3, 5, 8, 9]	5
empilha(2)	exceção: Erro: pilha cheia	[1, 3, 5, 8, 9]	5

Parte 3.1 - `inverte("PUCMINAS")`: empilha P, U, C, M, I, N, A, S (topo = S). Desempilhando: S, A, N, I, M, C, U, P, o que forma **"SANIMCUP"**.

Parte 3.2 - `balanceado(expr)`:

Expressão	Pilha durante a leitura	Resultado
(a+b)*[c-d]	(empilha;) desempilha (e combina; [empilha;] desempilha [e combina; pilha vazia no fim	true
{[()]}	empilha {, [, (;) combina com (;] combina com [; } combina com {; pilha vazia	true
((a)	empilha (, (;) combina com (; sobra um (na pilha no fim	false
a+b)	lê) com a pilha vazia, então não há abertura correspondente	false
(])]	empilha (, [; lê) e desempilha [, que não combina com)	false